

Module 06 [Data Structure with STL] (for Week 7)

Topic 1 [STL: stack class]

Problem statement: Write a menu program to create and manipulate Stack using stack class and perform the following operations using the specified method.

- i) use push() method to push data
- ii) use pop() method to pop data
- iii) use top() method to display top element
- iv) use empty() method to check whether stack is empty or not

```
** Stack **  
1. Push  
2. Pop  
3. Display()  
4. Exit  
Enter your option:
```

Code:

```
#include<bits/stdc++.h>  
using namespace std;  
  
int main(){  
    stack<int> st;  
    int opt, val;  
  
    while(true){  
        cout<<"\n1.Push\n2.Pop\n3.Top\n4.Check Empty\n5.Exit\n";  
        cin>>opt;  
  
        switch(opt){  
            case 1:  
                cout<<"Enter value: ";  
                cin>>val;  
                st.push(val);  
                break;  
  
            case 2:  
                if(!st.empty()){  
                    cout<<"Popped: "<<st.top()<<endl;  
                    st.pop();  
                } else cout<<"Stack Empty\n";  
                break;  
  
            case 3:  
                if(!st.empty())  
                    cout<<"Top: "<<st.top()<<endl;  
                break;  
  
            case 4:  
                if(st.empty())  
                    cout<<"Stack is Empty\n";  
                break;  
  
            case 5:  
                return 0;  
        }  
    }  
}
```

```

        else cout<<"Stack Empty\n";
        break;

    case 4:
        cout<<(st.empty()?"Empty":"Not Empty")<<endl;
        break;

    case 5:
        return 0;
    }
}
}

```

Topic 2 [STL: queue class]

Problem statement: Write a menu program to create and manipulate Queue using queue class. Perform the following operations using the specified method.

- i) use push() method to enqueue data
- ii) use pop() method to dequeue data
- iii) use front() method to display front element
- iv) use back() method to display rear element
- v) use empty() method to check whether queue is empty or not

```

** Queue **
1. Enqueue
2. Dequeue
3. Display Front
4. Display Rear
5. Exit
Enter your option:

```

Code:

```

#include<bits/stdc++.h>
using namespace std;

int main(){
    queue<int> q;
    int opt, val;

    while(true){
        cout<<"\n1.Enqueue\n2.Dequeue\n3.Front\n4.Rear\n5.Exit\n";
        cin>>opt;

        switch(opt){
            case 1:
                cin>>val;
                q.push(val);
                break;

```

```

    case 2:
        if(!q.empty()){
            cout<<"Removed: "<<q.front()<<endl;
            q.pop();
        } else cout<<"Queue Empty\n";
        break;

    case 3:
        if(!q.empty())
            cout<<"Front: "<<q.front()<<endl;
        break;

    case 4:
        if(!q.empty())
            cout<<"Rear: "<<q.back()<<endl;
        break;

    case 5:
        return 0;
    }
}
}

```

Topic 3 [STL: vector class]

Problem statement: Write a menu program to create and manipulate linked list using vector class and use the following methods

- i) **push_back()**
- ii) **push_front()**
- iii) **begin()**
- iv) **end()**

- v) **pop_back()**
- vi) **pop_front()**
- vii) **insert()**
- viii) **erase()**

```

** Linked List **
1. Insert
2. Delete
3. Update
4. Search
5. Exit
Enter your option:

```

Code:

```

#include<bits/stdc++.h>
using namespace std;

int main(){
    vector<int> v;
    int opt, val, pos;

    while(true){
        cout<<"\n1.Insert\n2.Delete\n3.Update\n4.Search\n5.Exit\n";
        cin>>opt;

        switch(opt){
            case 1:
                cin>>val;
                v.push_back(val);
                break;

            case 2:
                cin>>pos;
                if(pos>=0 && pos<v.size())
                    v.erase(v.begin()+pos);
                break;

            case 3:
                cin>>pos>>val;
                if(pos>=0 && pos<v.size())
                    v[pos]=val;
                break;

            case 4:
                cin>>val;
                for(int i=0;i<v.size();i++)
                    if(v[i]==val)
                        cout<<"Found at "<<i<<endl;
                break;
        }
    }
}
```

```
        case 5:  
            return 0;  
    }  
}  
}
```

Topic 3.1 [STL: vector class]: Add more feature to the menu of Topic 3.

Module 07 [UML+Java Basics]

(for Week 8: 19-23/7/2025)

Topic 1 [UML]

Problem statement: Draw the UML diagram for the following class

```
class Test{
    private: int x;
    protected: int y;
    public: int z;
    public:
        void SetData(int a,int b,int c){
            x=a;y=b;z=c;
        }
        float getAverage(){
            Return (x+y+z)/3.0;
        }
}
```

Code:

| **Test** |

| - x : int |

| # y : int |

| + z : int |

| + SetData(a:int, |

| b:int, |

| c:int) |

| + getAverage():float |

Topic 2 [UML: Aggregation]

Problem statement: Create two classes Doctor and Patient to store the information of doctors and patients respectively. Now write codes to create association between these two classes by aggregation.

[note: if Patient class is destroyed then Doctor class is still exist]

class Doctor{ //write your code }	class Patient{ //write your code }	int main(){ //write your code }
---	--	---------------------------------------

Code:

```
#include<bits/stdc++.h>
using namespace std;
class Student
{
    public:
        str#include<bits/stdc++.h>
using namespace std;

class Patient{
public:
    string name;
    Patient(string n){ name = n; }
};

class Doctor{
public:
    Patient* p; // aggregation (reference)

    Doctor(Patient* p){
        this->p = p;
    }

    void show(){
        cout<<"Patient: "<<p->name<<endl;
    }
};

int main(){
    Patient p("Ragib");
    Doctor d(&p);

    d.show();
}ing name;
Student(string n)
{
    name = n;
}
```

```
};
```

Topic 3 [UML: Composition]

Problem statement: Create two classes Doctor and Patient to store the information of doctors and patients respectively. Now write codes to create association between these two classes by composition.

[note: if Doctor class is destroyed then Patient class is also destroyed]

class Doctor{ //write your code }	class Patient{ //write your code }	int main(){ //write your code }
---	--	---------------------------------------

Code:

```
#include<bits/stdc++.h>
using namespace std;

class Patient{
public:
    string name;
    Patient(string n){ name=n; }
};

class Doctor{
public:
    Patient p; // composition

    Doctor(string name): p(name){}

    void show(){
        cout<<"Patient: "<<p.name<<endl;
    }
};

int main(){
    Doctor d("Ragib");
    d.show();
}
```

Topic 4: [java Programming Basics]

Problem Statement: Write java programs to do the following

- i) Display your name and address in two lines
- ii) Take two integers as inputs and prints the bigger one
- iii) Input a series of 10 numbers into an array ax[] and finds the largest, smallest and average of these numbers. Also write code for searching and sorting.

Code(i):

```
class A{
```



```

    public static void main(String[] args){
        System.out.println("Ragib Ishrak");
        System.out.println("Bangladesh");
    }
}
Code(ii):
import java.util.*;
class A{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        int a=sc.nextInt(), b=sc.nextInt();
        System.out.println(Math.max(a,b));
    }
}
Code(iii): import java.util.*;
class A{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        int a=sc.nextInt(), b=sc.nextInt();
        System.out.println(Math.max(a,b));
    }
}

```

```

PS D:\Satu\1-2\CSE
" ; if ($?) { java
Ragib Ishrak
Bangladesh

```

```

PS D:\Satu\1-2\CSE 120
($?) { javac A.java }
17 45
45

```

```

PS D:\Satu\1-2\CSE 1204\labmodule6> cd "d:\Satu\1-2\CSE 1204\labmodule6\" ; if
($?) { javac main.java } ; if ($?) { java main }
Error: Could not find or load main class main
Caused by: java.lang.ClassNotFoundException: main
PS D:\Satu\1-2\CSE 1204\labmodule6>

```

Topic 5 [java Programming private method]

Problem Statement: Verify that private methods can be used only inside the class but it cannot be used from outside the class.

Topic 6 [java Programming static method/member]

Problem Statement: Verify that static methods can access static method/member but cannot access to non-static method/member.

Topic 7 [java Programming]

Problem Statement: Write a Java Program to initialize radius of a circle using constructor/Setter and calculate area of the circle.